

Jenson USA Data Analysis

A comprehensive SQL-driven analysis of customer behavior, staff performance, inventory management, and store operations

Presented by: Krishna Agarwal
Cohort 6



Project Background

Business Intelligence Initiative

This project delivers critical insights into Jenson USA's operations by analyzing structured business data across customer interactions, sales activities, inventory control, and store performance metrics. Using SQL as our primary analytical tool, we transform raw transactional data into actionable intelligence that drives strategic decision-making and operational excellence.

Project Objectives



Customer Behavior Analysis

Understand purchasing patterns, preferences, and customer lifetime value



Staff Performance Evaluation

Measure sales effectiveness and identify top performers



Inventory Management

Monitor stock levels and optimize product availability



Store Operations Assessment

Evaluate store-wise performance and operational efficiency

These insights empower management to make data-driven decisions that enhance profitability and customer satisfaction.

The Power of SQL Analytics

SQL transforms complex relational data into clear, actionable business intelligence—enabling us to extract meaningful patterns from millions of transactions.

By leveraging structured query language, we can efficiently join multiple data sources, aggregate metrics, and uncover trends that would be impossible to identify manually.

Dataset Architecture

The Jenson USA dataset employs a normalized relational database structure, where interconnected tables work together to provide a complete view of business operations.

Brands & Categories

Product classification

Stocks & Stores

Location inventory

Staffs

Employee records



Customers

Buyer information

Orders & Order Items

Transaction details

Products

Inventory catalog

Data Table Overview

Product & Classification Tables

- **Brands:** Manufacturer and brand identity data
- **Categories:** Product grouping and hierarchy
- **Products:** Complete catalog with pricing and specifications

Transaction Tables

- **Orders:** Purchase header information
- **Order_Items:** Line-level transaction details

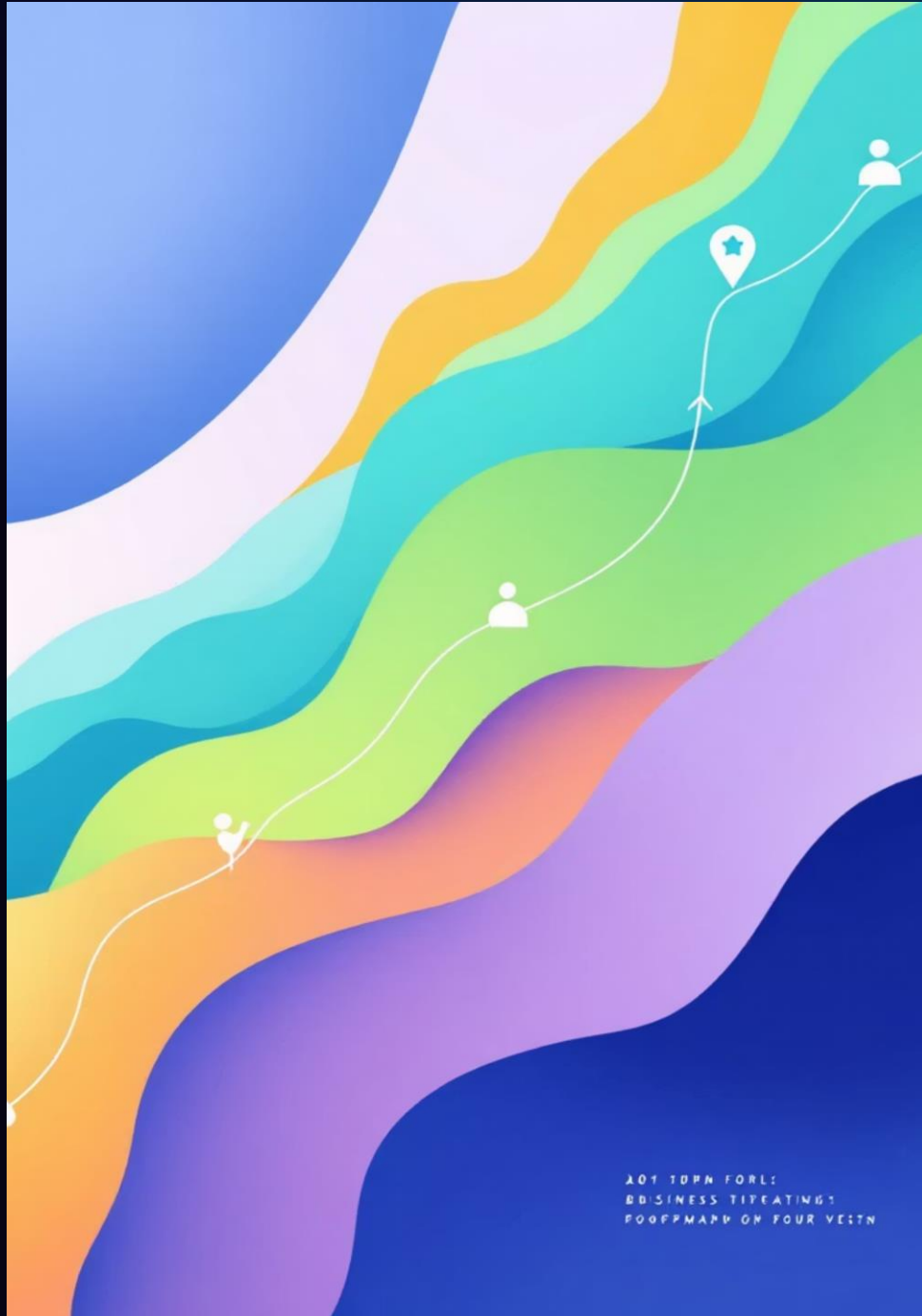
Customer & Staff Tables

- **Customers:** Demographic and contact information
- **Staffs:** Employee data and roles

Inventory & Location Tables

- **Stocks:** Product availability by location
- **Stores:** Store locations and operational data

These nine interconnected tables create a comprehensive ecosystem for analyzing every aspect of Jenson USA's business operations.



Project Roadmap

- 1 Data Extraction**
Query and aggregate data from nine relational tables
- 2 Analysis Execution**
Apply SQL techniques to generate insights across four key areas
- 3 Insights Delivery**
Present actionable recommendations to stakeholders
- 4 Implementation**
Support management in applying findings to business strategy

Next steps: Execute comprehensive SQL queries to unlock the full potential of Jenson USA's business data.

Expected Business Impact



Revenue Growth

Data-driven insights help identify opportunities to increase sales and improve profit margins.



Operational Efficiency

Optimize inventory levels and staff allocation based on performance data.



Customer Satisfaction

Better understand customer needs to enhance service quality and retention.

This analysis framework transforms raw data into strategic advantages, positioning Jenson USA for sustained competitive success in the cycling retail market.



Relational Database Benefits



Data Integrity

Relationships ensure consistency across all business records



Query Flexibility

Join tables to answer complex business questions



Scalable Analysis

Efficiently process large datasets for insights

1. Find the total number of products sold by each store along with the store name.

```
select stores.store_name, sum(order_items.quantity)
From stores
JOIN Orders
ON Stores.store_id = orders.store_id
JOIN order_items
on order_items.order_id = orders.order_id
group by stores.store_name
```

2. Calculate the cumulative sum of quantities sold for each product over time.

```
• SELECT
    products.product_id,
    products.product_name,
    orders.order_date,
    order_items.quantity,
    SUM(order_items.quantity)
      OVER (
        PARTITION BY products.product_id
        ORDER BY orders.order_date
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
      ) AS cumulative_quantity
FROM products
JOIN order_items
  ON products.product_id = order_items.product_id
JOIN orders
  ON order_items.order_id = orders.order_id
ORDER BY products.product_id, orders.order_date;
```

3. Find the product with the highest total sales (quantity * price) for each category.

```
• SELECT
    categories.category_name,
    products.product_name, SUM(order_items.quantity * products.list_price) AS total_sales
FROM products
JOIN order_items
    ON products.product_id = order_items.product_id
JOIN categories
    ON products.category_id = categories.category_id
GROUP BY
    categories.category_id,
    products.product_id
HAVING SUM(order_items.quantity * products.list_price) = (
    SELECT MAX(product_sales)
    FROM (SELECT SUM(order_items.quantity * products.list_price) AS product_sales
          FROM products JOIN order_items
              ON products.product_id = order_items.product_id
          WHERE products.category_id = categories.category_id
          GROUP BY products.product_id)x);
```


4. Find the customer who spent the most money on orders.

```
• SELECT
    customers.first_name,
    customers.last_name,
    SUM(order_items.quantity * products.list_price) AS total_spent
FROM customers
JOIN orders
    ON customers.customer_id = orders.customer_id
JOIN order_items
    ON orders.order_id = order_items.order_id
JOIN products
    ON order_items.product_id = products.product_id
GROUP BY Customers.first_name, customers.last_name
ORDER BY
    total_spent DESC
LIMIT 1;
```

5. Find the highest-priced product for each category name.

```
• SELECT
    categories.category_name,
    products.product_name,
    products.list_price
FROM products
JOIN categories
    ON products.category_id = categories.category_id
WHERE products.list_price = (
    SELECT MAX(products.list_price)
    FROM products
    WHERE products.category_id = categories.category_id);
```


6. Find the total number of orders placed by each customer per store.

```
• SELECT
    customers.first_name,
    customers.last_name,
    stores.store_id,
    COUNT(orders.order_id) AS total_orders
FROM customers
JOIN orders
    ON customers.customer_id = orders.customer_id
JOIN stores
    ON orders.store_id = stores.store_id
GROUP BY
    customers.first_name,
    customers.last_name,
    stores.store_id
ORDER BY stores.store_id;
```

7. Find the names of staff members who have not made any sales.

```
• SELECT
    staffs.staff_id,
    staffs.first_name,
    staffs.last_name
FROM staffs
LEFT JOIN orders
    ON staffs.staff_id = orders.staff_id
WHERE orders.order_id IS NULL;
```

8. Find the top 3 most sold products in terms of quantity.

```
• SELECT
    products.product_id,
    products.product_name,
    SUM(order_items.quantity) AS total_quantity_sold
FROM products
JOIN order_items
    ON products.product_id = order_items.product_id
GROUP BY
    products.product_id,
    products.product_name
ORDER BY
    total_quantity_sold DESC
LIMIT 3;
```

9. Find the median value of the price list.

```
• Select avg(list_price) As median_price
  FrOM (Select
    list_price,
    row_number() Over (order by list_price) As row_num,
    count(*) Over () As total_rows
  FrOm products) ranked
  Where row_num In (
    FLOOR((total_rows + 1) / 2),
    CEIL((total_rows + 1) / 2));
```


10. List all products that have never been ordered.(use Exists)

```
SELECT
    products.product_id,
    products.product_name
FROM products
WHERE NOT EXISTS (
    SELECT *
    FROM order_items
    WHERE order_items.product_id = products.product_id);
```


11. List the names of staff members who have made more sales than the average number of sales by all staff members.

```
• SELECT
    staffs.first_name,
    staffs.last_name
FROM staffs
JOIN orders
    ON staffs.staff_id = orders.staff_id
GROUP BY
    staffs.first_name,
    staffs.last_name
HAVING COUNT(orders.order_id) > (
    SELECT AVG(order_count)
    FROM (
        SELECT COUNT(order_id) AS order_count
        FROM orders
        GROUP BY staff_id) avg_sales);
```

12. Identify the customers who have ordered all types of products (i.e., from every category).

```
• SELECT
    customers.first_name,
    customers.last_name
FROM customers
WHERE NOT EXISTS
    (SELECT categories.category_id
     FROM categories WHERE NOT EXISTS (
         SELECT *
         FROM orders
         JOIN order_items
             ON orders.order_id = order_items.order_id
         JOIN products
             ON order_items.product_id = products.product_id
         WHERE
             orders.customer_id = customers.customer_id
             AND products.category_id = categories.category_id)); •
```

Thank you